

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

Structures de données et sous-algorithmes

Module 1 : Algorithmique

Ce TD de synthèse mobilise l'ensemble des notions du module (structures complexes, modularité, portée des variables, passage de paramètres) pour résoudre des problèmes algorithmiques complets de niveau Baccalauréat.

EXERCICE 1 Gestion d'une bibliothèque : conception modulaire

On souhaite concevoir un algorithme de gestion simplifiée d'une bibliothèque. Un ouvrage est caractérisé par : un code ISBN (chaîne de 13 caractères), un titre (chaîne), un auteur (chaîne), une année d'édition (entier) et un état (chaîne : "Disponible", "Emprunté", "Réservé"). La bibliothèque contient au maximum 500 ouvrages.

1. Proposer une déclaration de type `Enregistrement` pour un ouvrage et une structure de données adaptée pour stocker la collection. Justifier le choix d'un tableau à une dimension plutôt qu'un enregistrement unique.
2. Écrire la signature (en-tête) d'une fonction `RechercherParISBN` qui prend en paramètre la collection, sa taille effective et un ISBN, et qui renvoie l'indice de l'ouvrage ou -1 s'il est absent. Préciser le mode de passage des paramètres (par valeur ou par référence) pour la collection et justifier votre choix en termes de performance et de sécurité.
3. Écrire l'algorithme complet de la procédure `EmprunterOuvrage` qui modifie l'état d'un ouvrage identifié par son ISBN en "Emprunté" s'il est "Disponible", et affiche un message d'erreur adapté dans les autres cas (introuvable, déjà emprunté, réservé). Cette procédure doit appeler `RechercherParISBN`.
4. On ajoute une variable globale `NbTotalEmprunts` (entier) initialisée à 0, incrémentée à chaque emprunt réussi. Expliquer pourquoi l'utilisation d'une variable globale est ici pertinente (ou non) par rapport à un passage par référence d'un compteur. Discuter de l'impact sur la réutilisabilité de la procédure.
5. Proposer l'en-tête et décrire le rôle d'une procédure `AfficherDisponibles` qui parcourt la collection et affiche les détails des ouvrages disponibles. Comment garantir que cette procédure ne modifie pas accidentellement les données de la collection ?

EXERCICE 2 Analyse et correction d'un algorithme de tri modulaire

On étudie l'algorithme suivant, censé trier un tableau d'entiers par ordre croissant en utilisant une approche modulaire.

Variables globales

T : Tableau [1..N] d'entiers

```
N : Entier (* Taille effective *)

Algorithme Principal
  Afficher("Entrez N : "); Lire(N)
  Pour i de 1 à N faire
    Afficher("T[" , i , "]" : ); Lire(T[i])
  FinPour
  Appeler TrierTableau(T, N)
  Appeler AfficherTableau(T, N)
Fin

Procédure TrierTableau(Param Tab : Tableau d'entiers, Param Taille : Entier)
  Variables locales
    i, j, min, temp : Entiers
  Debut
    Pour i de 1 à Taille - 1 faire
      min <- i
      Pour j de i + 1 à Taille faire
        Si Tab[j] < Tab[min] Alors min <- j FinSi
      FinPour
      Si min <> i Alors
        temp <- Tab[i]
        Tab[i] <- Tab[min]
        Tab[min] <- temp
      FinSi
    FinPour
  Fin

Procédure AfficherTableau(Param Tab : Tableau d'entiers, Param Taille : Entier)
  Variables locales
    k : Entier
  Debut
    Pour k de 1 à Taille faire
      Afficher(Tab[k], " ")
    FinPour
    Afficher("\n")
  Fin
```

1. Identifier le type de tri implémenté dans `TrierTableau`. Donner sa complexité temporelle dans le pire cas en notation $O(\cdot)$.
2. Analyser le passage des paramètres `Tab` et `Taille` dans les deux procédures. S'agit-il de paramètres formels ou effectifs? Quel est le mécanisme de passage implicite pour `Tab` (valeur ou référence)? Quelles seraient les conséquences si `Tab` était passé par valeur?

3. La variable `N` est globale. La procédure `TrierTableau` déclare une variable locale `i`. Que se passe-t-il si la boucle principale du `Principal` utilisait aussi une variable `i` (globale) pour l'initialisation ? Expliquer le mécanisme de masquage (shadowing) et les risques d'erreurs.
4. Proposer une réécriture fonctionnelle (sans variable globale `T`) où le tableau est déclaré localement dans le principal et passé aux sous-algorithmes. Écrire les nouveaux en-têtes de `TrierTableau` et `AfficherTableau` en précisant explicitement les modes de passage (entrée, sortie, entrée/sortie) pour chaque paramètre formel.
5. Critiquer l'interface de `AfficherTableau` : pourquoi est-il préférable de passer `Taille` en paramètre plutôt que d'utiliser la borne supérieure statique du type tableau (ex : `NMAX`) ?

EXERCICE 3 Polynômes : structures imbriquées et opérations

Un polynôme $P(x) = a_n x^n + \dots + a_1 x + a_0$ de degré ≤ 20 est représenté par un enregistrement contenant son degré effectif (entier) et un tableau de 21 réels pour les coefficients (indice 0 pour a_0).

1. Déclarer le type `Polynome` (enregistrement) et écrire une fonction `CreerPolynome` qui initialise un polynôme nul (degré -1 ou 0 selon convention, tous coefficients à 0.0) et le renvoie. Justifier le choix d'une fonction plutôt que d'une procédure pour cette initialisation.
2. Écrire la procédure `SaisirPolynome` qui permet à l'utilisateur de saisir le degré (vérifié ≤ 20) et les coefficients. Discuter du mode de passage du paramètre `P` (entrée/sortie) et de l'intérêt de passer le degré par référence ou de le lire dans l'enregistrement.
3. Concevoir une fonction `Evaluer(P, x)` qui calcule $P(x)$ pour un réel x donné en utilisant la méthode de Horner. Préciser les paramètres formels et leur mode. Pourquoi cette fonction ne doit-elle pas modifier son argument `P` ? Comment l'exprimer dans l'en-tête ?
4. Concevoir une procédure `Deriver(Var P : Polynome)` qui modifie `P` pour le remplacer par sa dérivée $P'(x)$. Détailler la mise à jour du degré et des coefficients. Que se passe-t-il si on dérive un polynôme constant ? Comment le gérer ?
5. Problème ouvert : On souhaite ajouter l'addition de deux polynômes. Comparer deux approches :

- Procédure `Additionner(Var P1, P2, PRes : Polynome)`
- Fonction `Additionner(P1, P2 : Polynome) : Polynome`

Discuter des avantages/inconvénients de chaque approche en termes de lisibilité, gestion mémoire (copie de structure), et programmation fonctionnelle vs impérative. Laquelle recommanderiez-vous dans un contexte de calcul formel intensif et pour-

quoi?

EXERCICE 4 Refactorisation : du monolithique au modulaire

L'algorithme suivant calcule les statistiques (moyenne, médiane, écart-type) d'une série de notes (réelles, max 100 élèves) saisies par l'utilisateur, le tout dans un bloc unique sans sous-algorithmes.

Algorithme StatsMonolithique

Variables

Notes : Tableau[1..100] de Reels

N, i, j : Entiers

Somme, Moyenne, Mediane, EcartType, Variance, Temp : Reels

Debut

// Saisie

Repetier Afficher("N (1..100) : "); Lire(N) JusquA (N>=1 Et N<=100)

Pour i de 1 à N faire Lire(Notes[i]) FinPour

// Tri à bulles (pour médiane)

Pour i de 1 à N-1 faire

 Pour j de 1 à N-i faire

 Si Notes[j] > Notes[j+1] Alors Temp<-Notes[j]; Notes[j]<-Notes[j+1]; Notes[j+1]<-Temp

 FinPour

FinPour

// Calculs

Somme <- 0; Pour i de 1 à N faire Somme <- Somme + Notes[i] FinPour

Moyenne <- Somme / N

Si N mod 2 = 1 Alors Mediane <- Notes[(N+1)/2]

Sinon Mediane <- (Notes[N/2] + Notes[N/2+1]) / 2 FinSi

Variance <- 0; Pour i de 1 à N faire Variance <- Variance + (Notes[i]-Moyenne)² FinPour

Variance <- Variance / N; EcartType <- Racine(Variance)

// Affichage

Afficher("Moyenne : ", Moyenne, " Mediane : ", Mediane, " Ecart-Type : ", EcartType)

Fin

1. Identifier au minimum 4 sous-tâches logiques distinctes dans cet algorithme qui mériteraient d'être encapsulées en sous-algorithmes (procédures ou fonctions).
2. Pour chaque sous-tâche identifiée, proposer un en-tête complet (nom, paramètres formels avec modes entrée/sortie, type de retour si fonction). Justifier le choix entre procédure et fonction pour le calcul de la médiane.
3. Réécrire l'algorithme principal uniquement sous forme d'appels à ces sous-algorithmes, en déclarant les variables nécessaires localement au principal. Montrer comment le tableau `Notes` et sa taille `N` sont propagés.

4. Le tri à bulles modifie le tableau `Notes` (effet de bord). Est-ce acceptable pour le calcul de la médiane ? Proposer une stratégie alternative (copie du tableau, tri sur indices, ou tri non destructeur) et discuter de son coût algorithmique.
5. Question de synthèse : En quoi la décomposition modulaire réalisée facilite-t-elle la maintenance (ex : changement de formule de variance, ajout du mode, changement de tri) et les tests unitaires ? Illustrer par un exemple concret de modification.